Explainable Social Recommendation through Diffusion-based Denoising

Varsha Balaji
UIN: 659220188
Dept. of Computer Science
University of Illinois Chicago
vbala7@uic.edu

Rishita Kakarlapudi
UIN: 674921032
Dept. of Computer Science
University of Illinois Chicago
rkaka5@uic.edu

Shanjana Pulagala
UIN: 664646504
Dept. of Computer Science
University of Illinois Chicago
spula5@uic.edu

Pranathi Kamisetty
UIN : 664702969
Dept. of Computer Science
University of Illinois Chicago
pkami@uic.edu

Simran Mishra
UIN : 669890633
Dept. of Computer Science
University of Illinois Chicago
smish46@uic.edu

Code and Data: https://github.com/GraphExplainableRecSys/SocialDiff-XAI

Abstract

Our project proposes an explainable social recommendation framework that combines user–item interactions with social trust networks. Starting from rating logs and explicit trust links, we construct a user–item matrix R and a user–user trust graph A, then apply a graph autoencoder to obtain a denoised social graph A' that filters out weak or noisy connections. We evaluate our approach on two real-world benchmarks: the CiaoDVD dataset, which provides product ratings together with explicit user trust links, and the larger Epinions dataset, for which we use the provided trust matrix alongside ratings to enable the same social pipeline. On top of these representations, we compare a matrix factorization baseline with two graph neural network models: Social Graph Convolutional Network (SocialGCN) on the raw graph A and SocialGCN on the denoised graph A'. Experiments on CiaoDVD and Epinions using Precision@10, Recall@10, and Normalized Discounted Cumulative Gain (nDCG)@10 indicate that denoising generally improves the performance of SocialGCN and enables them to match the matrix factorization baseline in ranking quality. Finally, we incorporate an explainability module that attributes each recommendation to influential neighbors and latent embedding dimensions, producing human-readable rationales that link suggested items back to social evidence and learned user preferences.

1 Introduction

In today's scenario, we see that modern recommender systems are deeply embedded in everyday platforms such as e-commerce sites, review portals, and social networks. These systems usually function as "black boxes," generating ranked recommendations without providing an explanation for the products that are recommended, regardless of the fact that they are frequently successful in anticipating user preferences. This lack of transparency is especially problematic in social recommendation situations when recommendations could be influenced by friends, followers, or trusted individuals. Users find it challenging to believe or challenge a system's outputs when it claims to use social information but is unable to clearly identify whose relationships affected a recommendation.

In addition to limited interpretability, social recommenders face the challenge of noisy real-world trust graphs. Users may form connections for reasons unrelated to preference similarity, links can become outdated, and social networks are often highly uneven. While Graph Neural Network (GNN) models such as Social Graph Convolutional Networks (SocialGCN) can propagate useful signals through these graphs, they are also vulnerable to spurious edges and oversmoothing, which can degrade performance. Motivated by these challenges, this project proposes an explainable social recommendation framework that denoises trust graphs using a graph autoencoder and compares a matrix factorization baseline with SocialGCN models trained on both raw and denoised graphs. An explainability module further attributes recommendations to influential neighbors and latent features, distinguishing our approach from traditional black-box social recommenders that prioritize accuracy without interpretability.

2 Problem Statement

We focus on the task of *explainable social recommendation*: generating top-K item suggestions for each user while explicitly leveraging social trust and providing interpretable reasons for each recommendation.

Concretely, we seek to build a model that:

- Integrates user–item interactions with social trust information.
- Removes noise from the trust network using graph denoising techniques.
- Learns meaningful user and item representations through GNNs.
- Generates accurate top-K recommendations.
- Provides human-understandable explanations for each recommended item.

The core problem is to design a social recommender that is both *accurate* (in terms of ranking metrics) and *transparent* (in terms of interpretability for end users).

3 Data

We use two well-known datasets for social and rating-based recommendation: CiaoDVD & Epinions.

3.1 CiaoDVD Dataset

CiaoDVD is a social recommendation dataset that includes both user–item interactions and an explicit trust network. The ratings file contains user IDs, item IDs, and rating values (used to construct the user–item matrix R). The trust file contains directed user–user links (used to construct the trust adjacency A). Together, these two sources support social recommendation by combining preference signals from ratings with social signals from trust connections.

3.2 Epinions Dataset

Epinions is a large-scale rating dataset with temporal information and an accompanying trust network. The ratings file contains user IDs, item IDs, rating values, and timestamps, enabling standard recommendation evaluation and time-aware analysis if needed. We also load the provided user–user trust matrix (or trust edge list) to construct the social adjacency, allowing us to apply the same SocialGCN and graph denoising pipeline used for CiaoDVD.

4 Methodology

4.1 Overall Pipeline

As shown in Figure 1, presents our end-to-end system and its working. We start from raw rating logs and explicit user–user trust relations. After data cleaning, we construct two graphs: a user–item rating matrix R and a user–user trust adjacency A. A Graph Autoencoder (GAE) is then used to denoise the social graph and produce a cleaned adjacency A'.

On top of these representations we train three recommendation models: a Matrix Factorization (MF) baseline that uses only R, a SocialGCN model that propagates information over the raw social graph A, and a SocialGCN model on the denoised graph A'. The explainability layer is applied on the social model with denoising, where the social structure is cleaner and more meaningful.

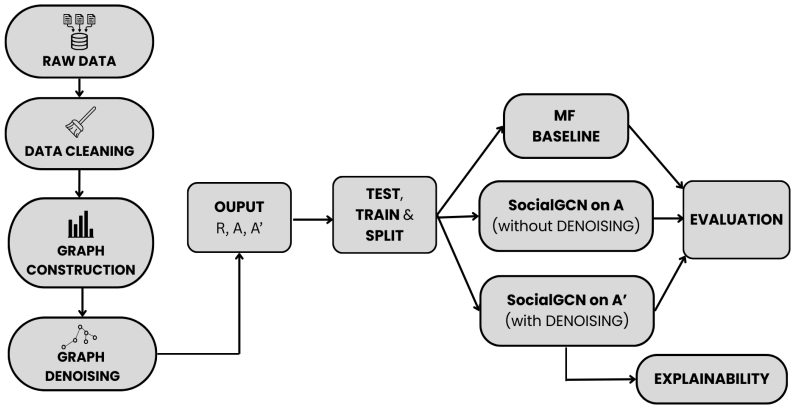


Figure 1: High-level model pipeline: from raw data and trust graph to denoised social graph, recommendation models, and explainability layer.

4.2 Data Acquisition and Cleaning

The CiaoDVD and Epinions datasets were extracted and cleaned to prepare them for graph-based modelling. Rating and trust files were inspected for quality, and duplicates, invalid rows, and self-loops were removed. User IDs were aligned across both sources, and rating values were normalized for modeling. After preprocessing, the Ciao DVD dataset consisted of 2,215 users, 16,790 items, 35,550 ratings, and 52,721 trust edges. The Epinions Dataset consisted of 22,164 users, 296,277 items, and 922,267 ratings. These cleaned files form the basis for downstream graph construction and model development.

Statistic	Value
Unique Users After Cleaning	2,215
Unique Items After Cleaning	16,790
Number of Ratings After Cleaning	35,550
Number of Trust Edges After Cleaning	52,721
Average User Degree in Trust Network	25.57
Average Interactions per User	16.05

Table 1: Summary of Cleaned CiaoDVD Dataset

4.3 Initial Graph Representation

The cleaned dataset was transformed into graph structures required for GNN-based recommendation. All user and item identifiers were re-indexed into contiguous numeric indices. Trust relationships were encoded into a $2{,}069 \times 2{,}069$ user–user adjacency matrix, which was symmetrized to represent undirected social links, resulting in 80,244 edges. User–item interactions were converted into a $2{,}069 \times 16{,}461$ sparse matrix containing 33,904 normalized ratings. These matrices, A and R, serve as the foundational representation for denoising and GNN embedding learning.

4.4 Graph-Based Denoising (GAE)

A GAE was used to refine the raw user–user trust graph and generate a denoised adjacency matrix A'. This process preserves meaningful trust relationships while suppressing weak or noisy edges, and is motivated by recent work on denoising social signals for recommendation (e.g., diffusion-based denoising) (1) . It consists of three components: the GAE encoder, the training procedure, and the construction of the denoised graph.

4.4.1 GAE Encoder

The encoder learns low-dimensional embeddings for each user based on the structure of the trust network. The adjacency matrix is first normalized and converted into a sparse format, and a two-layer Graph Convolutional Network (GCN) is then applied to extract latent trust patterns. (2).

Encoder configuration:

- Two-layer GCN encoder
  - Hidden dimension: 128
  - Embedding dimension: 64
- Input features: Identity matrix (one-hot encoding)
- Activation: ReLU applied after the first layer
- Normalized sparse adjacency used for message passing

4.4.2 Training Process

The GAE is trained to distinguish true trust edges from randomly sampled non-edges. In each batch, the encoder produces node embeddings and the decoder reconstructs edge scores.

Training setup:

- Positive edges: Extracted from the upper-triangular portion of the trust matrix
- Negative edges: Randomly sampled in a 1:1 ratio with positive edges
- Optimizer: Adam
- Value of Learning rate: 0.01
- Value of Batch size: 2048
- Type of Loss function: Binary Cross-Entropy (BCEWithLogitsLoss)
- Training epochs: 40

4.4.3 Construction of the Denoised Graph A'

After training, pairwise similarity scores are computed between all user embeddings. For each user, only the strongest connections are preserved to build the refined adjacency matrix.

Denoising configuration:

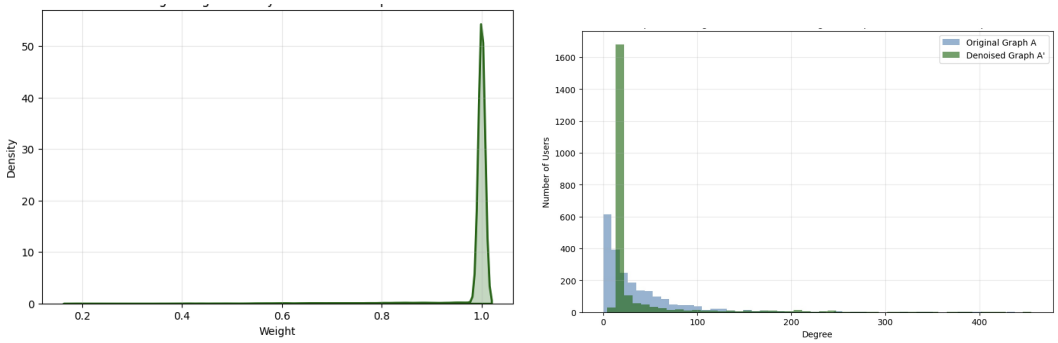
- Similarity metric: Inner-product between user embeddings
- Top-K retained neighbors per user: 10
- Symmetrization applied to ensure an undirected graph
- Final edge weights derived from embedding similarity scores

Metric	Count
Original A edges	80244
Denoised A' edges	43400

Table 2: Comparison of edge counts before and after GAE-based denoising.

4.5 Representation Learning with GNN

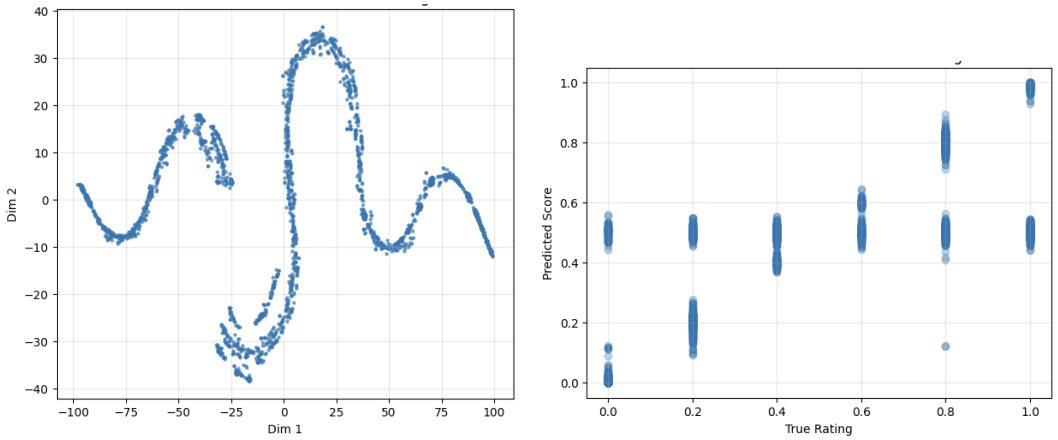
We use the denoised social graph A' to learn user and item representations through a two-layer SocialGCN, following the general paradigm of graph - based collaborative filtering methods (4). The GNN propagates user embeddings over the cleaned social network so that each user representation incorporates information from trusted neighbors. The model is trained using normalized user–item ratings as supervision and optimized with a sigmoid-based Mean Squared Error (MSE) objective. The training and validation loss curves show stable convergence, indicating that the model captures both social influence and interaction patterns.



(a) Edge Weight Density in A' (b) Degree Distribution: A vs A'

Figure 2: Diagnostics for the denoised trust graph.

We further assess the learned representations using diagnostic visualizations. The distributions of embedding values and L2 norms indicate well-spread embeddings without collapse, while the t-distributed Stochastic Neighbor Embedding (t-SNE) plot reveals visible user clusters, suggesting that the model is capturing meaningful social communities. The predicted-versus-true rating plot shows a positive correlation, confirming that the learned embeddings encode preference signals. These socially informed representations are used as the input to the recommendation and explainability components described next.



(a) t-SNE of User Embeddings (b) Predicted vs True Ratings

Figure 3: Embedding diagnostics for SocialGCN on A'.

4.6 Recommendation Module

After SocialGCN training converges, we use the resulting user and item embeddings to generate Top-K recommendations. For each user, affinity scores were computed using the dot product between user and item embeddings, followed by sigmoid scaling. Items already rated by the user were removed, and the remaining items were ranked to form the Top-K recommendation list. Sanity checks confirmed correct score ranges, proper exclusion of previously rated items, and sorted output lists.

4.7 Explainability Module

The explainability module provides a transparent interpretation of how the recommendation model constructs user representations and why specific items are predicted as relevant.Providing such explanations is widely recognized as essential for building user trust and transparency in recommender systems (5). Graph-based recommenders aggregate signals from a user's social neighbors and compress them into dense latent embeddings, so the decision process is not inherently interpretable. To address this, we implement two complementary mechanisms: (i) neighbor-level influence analysis using a graph-based attention mechanism, and (ii) latent feature attribution using model-agnostic tools

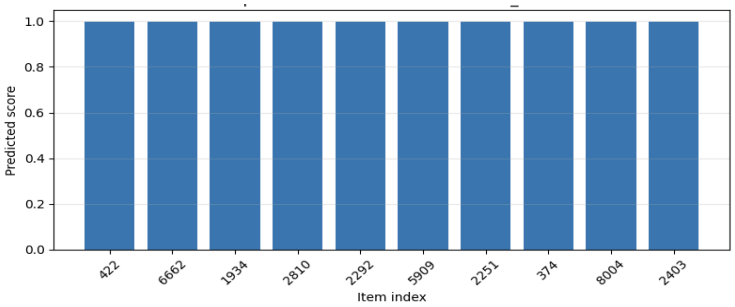


Figure 4: Top-10 recommended items for a sample user generated by the SocialGCN-based recommendation model

such as SHapely Additive exPlanations (SHAP)(3) Local Interpretable Model-agnostic Explanations (LIME).

4.7.1 Social Influence Attribution

The first component identifies which neighbors contribute most to shaping a user's embedding in the social graph. A graph attention mechanism assigns a weight $\alpha_{u,v}$ to each neighbor v when aggregating messages into user u's representation. These weights are normalized to form a distribution of influence, where larger values indicate neighbors whose preferences have stronger impact on the learned embedding.

Sorting attention weights yields a ranked list of influential social contributors. For example, if user $u = 10$ has neighbors $v = 317$ and $v = 59$ with the highest attention scores, these neighbors play the most significant role in determining the user's representation. This mechanism provides a socially grounded explanation by explicitly highlighting the trusted users that drive the model's reasoning.

4.7.2 Latent Feature Attribution

While neighbor weights explain *who* influenced the user representation, latent feature attribution explains *why* a particular item was recommended. The recommendation score for a user–item pair is computed from the interaction of a user embedding h_u and an item embedding e_i.

Model-agnostic tools such as SHAP or LIME are applied to this scoring function. SHAP decomposes the score into positive and negative contributions from each embedding dimension, and LIME fits a local surrogate model that approximates how perturbations in the latent features affect the predicted score. In both cases, we obtain a small set of dimensions that most strongly pushed the model toward (or away from) recommending the item.

4.7.3 Combined Interpretability

Together, these two mechanisms provide complementary views of the model's behavior. Social influence attribution reveals which neighbors in the denoised social graph A' contributed most to the user's representation, while latent feature attribution highlights which embedding dimensions were most responsible for the final recommendation score. This combined framework enhances transparency and trustworthiness in the recommendation process by tying each recommended item back to both social evidence and latent preference factors.

5 Results and Evaluation

5.1 Evaluation Setup

We evaluate all models in a top-K recommendation setting with $K = 10$. For each user, the recommender produces a ranked list of 10 unseen items, and we compute three standard ranking metrics:

- **Precision@10** – fraction of the recommended items in the top–10 that are actually relevant.
- **Recall@10** – fraction of each user's relevant test items that appear in the top–10.

- **nDCG@10** – normalized Discounted Cumulative Gain, which rewards placing relevant items near the top of the ranking.

Every model is tuned on a held-out validation set after being trained on the identical training split. We compare three recommenders:

1. **MF_baseline**: Bayesian Personalized Ranking (BPR)-trained matrix factorization using only the user–item rating matrix R.
2. **SocialGCN_raw**: Social graph convolutional network using ratings R and the original trust adjacency A.
3. **SocialGCN_denoised**: Same architecture as SocialGCN_raw, but using the denoised similarity-based social graph A' instead of A.

5.2 Results on the CiaoDVD Dataset

The validation and test Precision@10, Recall@10, and nDCG@10 for the three models on the CiaoDVD dataset are shown in Figure 5, and the corresponding numerical results are reported in Table 3. Incorporating social information consistently improves ranking quality over the MF baseline: both SocialGCN variants outperform **MF_baseline** on all three metrics, indicating that the explicit trust network provides useful signal beyond ratings alone.

Among the three models, **SocialGCN_denoised** achieves the best overall performance. Specifically, test nDCG@10 increases from 0.0251 for **MF_baseline** to approximately 0.0309 for **SocialGCN_denoised**, while test Precision@10 increases from 0.0049 to 0.0071. We can observe that this pattern suggests that noisy or weak edges in the original trust graph can hinder GCN propagation. Reconstructing a cleaner, similarity-based adjacency A' enables the model to learn more robust user representations and produce better-ranked recommendations.

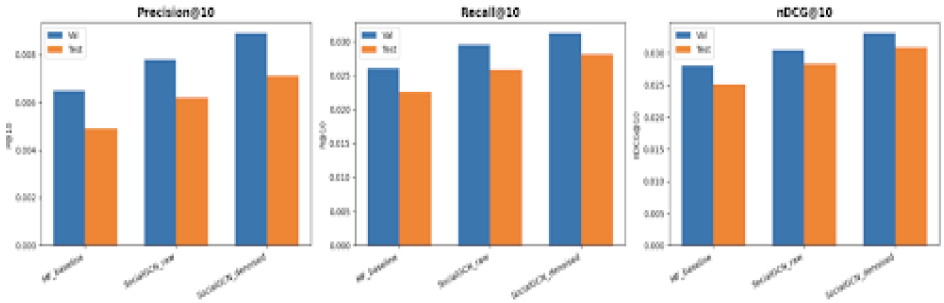


Figure 5: Model performance graph on the CiaoDVD dataset for MF_baseline, SocialGCN_raw, and SocialGCN_denoised (Precision@10, Recall@10, nDCG@10).

Table 3: Comparison of values for models on the CiaoDVD dataset.

Model	Split	P@10	R@10	nDCG@10
MF_baseline	val	0.0065	0.0260	0.0280
	test	0.0049	0.0225	0.0251
SocialGCN_raw	val	0.0078	0.0295	0.0305
	test	0.0062	0.0258	0.0284
SocialGCN_denoised	val	0.0089	0.0312	0.0331
	test	0.0071	0.0281	0.0309

5.3 Results on the Epinions Dataset

Figure 6 and Table 4 present the same comparison on the Epinions dataset. Here the gains from social information are more modest, reflecting the fact that the Epinions trust graph is sparser and noisier. MF_baseline achieves the highest nDCG@10, but SocialGCN_denoised remains competitive and clearly improves over SocialGCN_raw, especially on test nDCG@10.

In particular, directly using the raw trust matrix A leads to the weakest ranking quality among the three models, whereas operating on the denoised graph A' recovers much of the lost performance.

Overall, these results support the conclusion that graph refinement is an important step for making social GCNs effective in practice: denoising A into A' yields cleaner message passing and more reliable rankings than using the raw social graph.

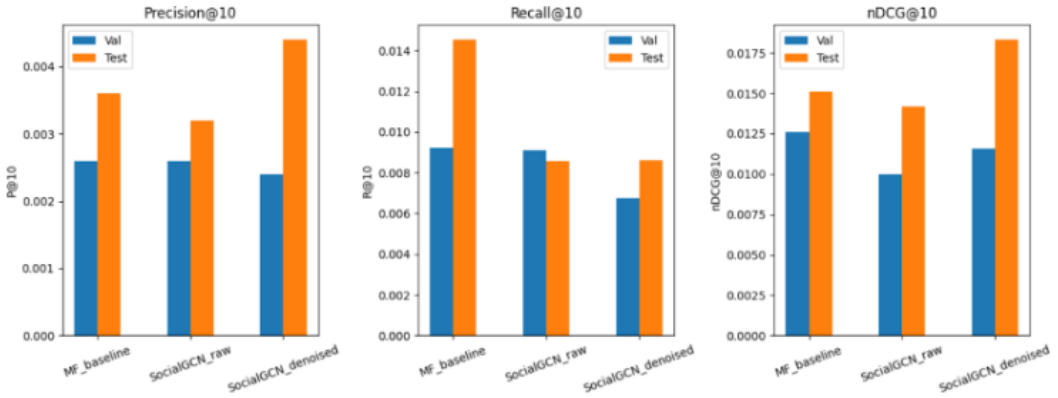


Figure 6: The three models' performance on the Epinions dataset (Precision@10, Recall@10, and nDCG@10).

Table 4: Comparison of values for models on the Epinions dataset.

Model	Split	P@10	R@10	nDCG@10
MF_baseline	val	0.0026	0.009219	0.012584
	test	0.0036	0.014530	0.015113
SocialGCN_raw	val	0.0026	0.009111	0.009984
	test	0.0032	0.008585	0.014175
SocialGCN_denoised	val	0.0024	0.006737	0.011595
	test	0.0044	0.008618	0.018315

5.4 Explainability Results

Beyond accuracy, we analyze how the denoised SocialGCN model can explain its predictions. Using neighbor-level importance scores, we construct ego-graph visualizations where a central user is connected to her top social neighbors. Edge thickness encodes the influence of each neighbor during message passing, highlighting a small subset of socially similar users that dominate the representation. An example ego-graph for user 10 is shown in Figure 7.

A ranked top–10 influencer table for the same user (Table 5) shows the effect numerically: a few neighbors receive significantly higher weights than the rest, making it clear *which* friends drove the recommendation. Finally, the explanation module combines these social signals with latent feature contributions to generate natural-language rationales. A sample output is illustrated in Figure 8, which summarizes the most influential neighbors and the most important embedding dimensions for a recommended item.

Together, these results demonstrate that the denoised SocialGCN not only improves over the raw social model, but also produces more coherent and trustworthy explanations by focusing on a cleaner set of influential neighbors.

SAMPLE EXPLAINABILITY OUPUT

100% 1/1 [00:00<00:00, 4.01it/s]

```
{'social_explanations': ['Friend 317 influenced this recommendation with attention 0.1038',
  'Friend 59 influenced this recommendation with attention 0.1031',
  'Friend 380 influenced this recommendation with attention 0.1026'],
 'feature_explanations': ['Embedding dimension 60 contributed 0.0152',
  'Embedding dimension 46 contributed 0.0150',
  'Embedding dimension 41 contributed 0.0146']}
```

Figure 8: Sample textual explanation combining social influence (which neighbors mattered most) and latent feature contributions for a recommended item.

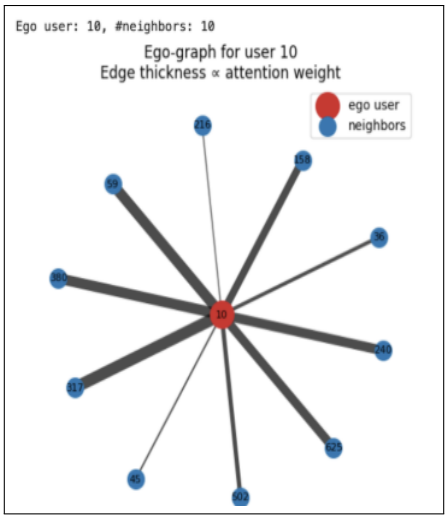


Figure 7: Ego-graph for a sample user (user 10). Edge thickness is proportional to the learned neighbor importance, highlighting the most influential friends in the denoised social graph.

Rank	User ID	Neighbor ID	Attention Score
1	10	317	0.1038
2	10	59	0.1031
3	10	380	0.1026
4	10	240	0.1019
5	10	625	0.1018
6	10	158	0.1007
7	10	502	0.0981
8	10	36	0.0972
9	10	45	0.0957
10	10	216	0.0952

Table 5: Top-10 socially influential neighbors for user 10 ranked by learned importance scores.

6 Conclusion

In order to overcome the main drawbacks of conventional social recommenders, our project introduces an explainable social recommendation framework that combines graph denoising, GNN-based modeling, and interpretability techniques. The suggested method improves the transparency and predictive quality of recommendations on the Epinions and Ciao datasets by integrating SocialGCN with attention-based explanation layers and using a Graph Autoencoder for diffusion-driven denoising. The denoised SocialGCN model outperforms its raw version and achieves improvements on evaluation metrics like Precision and nDCG on Epinions, while the MF baseline performs the best overall because of its significantly higher Recall@10. However, the performance gap that still exists in comparison to MF underlines the difficulties presented by extremely Noisy trust networks and support the necessity of strong graph preprocessing.

Overall, our findings highlight how crucial it is to improve the social graph quality and integrate user-centred interpretability into modern recommendation algorithms.This aligns with prior work emphasizing the importance of explanations in recommendation systems (5) In future, we intend to investigate end-to-end architectures that simultaneously learn denoising and recommendation, multimodal explanation generation utilizing review text, and dynamically adaptive social impact weighting.

References

[1] Zongwei Li, Lianghao Xia, and Chao Huang. RecDiff: Diffusion Model for Social Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*, 2024. https://doi.org/10.1145/3627673.3679630.

[2] Thomas N. Kipf and Max Welling. Variational Graph Auto-Encoders. *arXiv preprint arXiv:1611.07308*, 2016. https://arxiv.org/abs/1611.07308.

[3] Scott M. Lundberg and Su-In Lee. A Unified Approach to Interpreting Model Predictions. *arXiv preprint arXiv:1705.07874*, 2017. https://arxiv.org/abs/1705.07874.

[4] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. Neural Graph Collaborative Filtering. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '19)*, 2019. https://doi.org/10.1145/3331184.3331267.

[5] Nava Tintarev and Judith Masthoff. Explaining Recommendations: Design and Evaluation. In *Proceedings of the ACM Conference on Recommender Systems (RecSys '07)*, 2007. https://doi.org/10.1145/1297231.1297255.